

1. Заголовок

Кейс «Создание PWA-приложения для управления личными финансами» — предполагает разработку автономного (offline-first) прогрессивного веб-приложения для учёта доходов и расходов пользователя, анализа финансовых привычек и планирования бюджета.

Направление подготовки: 01.03.02 – Прикладная математика и информатика

Образовательная программа: Программная инженерия систем машинного обучения

Семестр: 2-й семестр

2. Аннотация

Кейс «Создание PWA-приложения для управления личными финансами» — это практическая задача по разработке современного веб-продукта, ориентированного на реального пользователя: приложение должно работать как «почти нативное», устанавливаться на устройство, запускаться из иконки, быть доступным без интернета и обеспечивать удобный ввод операций.

Учебная дисциплина: Ознакомительная практика

Актуальность и практическая значимость

Финансовая грамотность и регулярный контроль расходов — одна из наиболее востребованных «повседневных» цифровых практик. При этом многие пользователи сталкиваются с барьерами: неудобные интерфейсы, необходимость постоянного интернета, отсутствие приватности или сложный ввод данных. PWA-подход позволяет реализовать приложение, которое:

- работает **офлайн** (в дороге, в метро, без связи),
- быстро запускается и выглядит как мобильное,
- может хранить данные **локально** (повышая приватность),
- при наличии сервера — поддерживает **синхронизацию** между устройствами.

Форма выполнения: групповая.

Кейс направлен на формирование следующих ключевых компетенций

Профессиональные компетенции (hard skills):

- **Frontend/PWA разработка:** создание интерфейсов, работа с Service Worker, manifest, offline-cache, установка на устройство.
- **Проектирование хранения данных:** IndexedDB/SQLite (через WebAssembly/библиотеки), либо серверное хранилище (PostgreSQL/MongoDB) + локальный кэш.
- **Разработка API (опционально для облачной версии):** REST/GraphQL для синхронизации данных, авторизация, защита.
- **Архитектура клиента:** управление состоянием, маршрутизация, обработка ошибок, валидация форм.
- **DevOps (основы):** Git-процесс, деплой на публичный хостинг (например, Vercel/Netlify), базовые практики CI.

- **UI/UX (основы):** пользовательские сценарии, минимизация «трения» при вводе трат, доступность (a11y).

Универсальные компетенции (soft skills):

- **Командная работа:** распределение ролей, code review, совместное принятие решений.
- **Управление проектом:** планирование спринтов, бэклог, приоритизация.
- **Продуктовое мышление:** формирование MVP, фокус на ценности и удобстве пользователя.
- **Коммуникация и презентация:** регулярные демонстрации, аргументация решений.
- **Системное мышление:** целостный взгляд на продукт (данные → сценарии → интерфейс → надежность).

3. Постановка задания

Название кейса: Разработка PWA-приложения для учета личных финансов (MVP).

Цель выполнения кейса:

Сформировать и закрепить практические навыки проектирования и разработки автономного веб-приложения в итеративном формате. В ходе работы студенты осваивают принципы **offline-first**, а также (при выборе расширенного варианта) — клиент-серверную архитектуру и синхронизацию данных.

Конкретный образ результата (MVP):

Готовый MVP, развернутый на публичном хостинге, который предоставляет пользователю возможность:

1. **Добавлять операции** доходов и расходов: сумма, категория, дата/время, комментарий.
2. **Просматривать историю операций** (список, фильтры по периоду/категории, поиск по тексту).
3. **Вести категории** (минимальный набор + пользовательские, цвет/иконка — опционально).
4. **Получать простую аналитику:** баланс за период, сумма расходов по категориям, базовые графики.
5. **Работать офлайн:** приложение открывается без интернета; данные сохраняются локально.
6. **Устанавливаться как PWA:** наличие manifest, иконок, корректной установки на мобильное/desktop.
7. *(Опционально, как расширение)* **Аккаунт и синхронизация:** регистрация/вход, облачное хранение, перенос данных между устройствами.

4. Промежуточные показы (Milestones)

В рамках проекта предусмотрены **4 обязательные контрольные демонстрации**.

- **Показ 1 (конец 3-й недели):** защита концепции, UX-сценариев и архитектуры (offline-first).
- **Показ 2 (конец 6-й недели):** демонстрация ядра приложения: ввод операций и локальное хранение.

- **Показ 3 (конец 9-й недели):** категории, фильтры, базовая аналитика, улучшенный UX.
- **Показ 4 (конец 12-й недели):** демонстрация MVP целиком: PWA-установка + офлайн-режим + стабильность.

5. Форма и формат отчетности (финальная)

1. **Ссылка на работающий MVP** (публичный URL).
2. **Ссылка на Git-репозиторий** с полной историей коммитов.
3. **Техническая документация (README.md)** в репозитории:
 - архитектура и выбранный стек,
 - инструкция по локальному запуску,
 - описание PWA-части (manifest, service worker, offline-кеш),
 - описание модели данных и хранения (IndexedDB/сервер).
4. **Финальная презентация (10–15 слайдов)** и защита проекта.

6. Сроки выполнения

15 недель.

7. Методические рекомендации обучающимся

Рекомендуемый состав группы по ролям (для команды из 4 человек)

- **Team Lead / Project Manager:** планирование, бэклог, интеграция, контроль качества.
- **Frontend/PWA разработчик:** UI, state management, PWA (SW/manifest), офлайн-сценарии.
- **Backend разработчик (если есть синхронизация):** API, БД, авторизация, безопасность.
- **Тестировщик / QA (или инженер по качеству):** сценарии тестирования, чек-листы, регресс.

Если команда решает делать **полностью автономное** приложение без сервера, роль backend можно трансформировать в **Data/Architecture Engineer** (IndexedDB, миграции схемы, импорт/экспорт, качество данных).

Примерный план работы по спринтам (3 недели на спринт)

- **Спринт 1 (Недели 1–3): Планирование, UX и архитектура**
 - **Задачи:**
 - определить целевого пользователя и сценарии (быстрый ввод, просмотр статистики);
 - сформировать **User Stories** для MVP;
 - спроектировать **модель данных** (операции, категории, настройки);
 - выбрать стек (например, React/Vue + TypeScript; IndexedDB через Dexie; графики; UI-kit);
 - продумать PWA-требования: manifest, стратегия кэширования, офлайн-поведение;
 - подготовить wireframes основных экранов (список, добавление операции, аналитика, настройки);
 - настроить Git-процесс, доску задач.

- **Результат для Показа 1:** презентация концепции, user flow, архитектура, схема данных, макеты экранов, план работ.
- **Спринт 2 (Недели 4–6): Ядро — операции и хранение данных**
 - **Задачи:**
 - каркас приложения: роутинг, базовый UI, навигация;
 - добавление/редактирование/удаление операции;
 - локальное хранилище (IndexedDB), валидация ввода;
 - первичная подготовка PWA: manifest, иконки, базовый service worker.
 - **Результат для Показа 2:** пользователь добавляет операции, видит список; данные сохраняются локально и не исчезают после перезагрузки.
- **Спринт 3 (Недели 7–9): Категории, фильтры, аналитика**
 - **Задачи:**
 - справочник категорий (создание/редактирование);
 - фильтры по периоду/категории, поиск;
 - аналитика: баланс, расходы по категориям, график (минимум один вид визуализации);
 - улучшение UX: быстрый ввод, шаблоны/последние категории (по возможности).
 - **Результат для Показа 3:** полноценное ведение операций с фильтрацией и базовой аналитикой.
- **Спринт 4 (Недели 10–12): PWA-готовность и надежность**
 - **Задачи:**
 - корректная установка PWA, проверка Lighthouse;
 - офлайн-режим: кэш статики, понятные сообщения об ошибках, предсказуемое поведение;
 - стабильность: обработка крайних случаев, тестирование, исправление багов;
 - *(опционально)* синхронизация/аккаунт: базовый сервер и авторизация.
 - **Результат для Показа 4:** демонстрация полного MVP: установка + офлайн + ключевые сценарии без критических багов.
- **Спринт 5 (Недели 13–15): Стабилизация и подготовка к защите**
 - **Задачи:**
 - регресс-тесты, полировка интерфейса;
 - рефакторинг, повышение качества кода, документация;
 - финальный деплой;
 - подготовка презентации и демо-сценария.
 - **Результат:** готовый к защите проект и все отчетные материалы.

8. Методические рекомендации по оцениванию

Общая система оценивания:

Итоговая оценка формируется по **100-балльной шкале**.

Шкала перевода баллов:

- **75–100:** Отлично
- **55–74:** Хорошо
- **40–54:** Удовлетворительно
- **0–39:** Неудовлетворительно

Критерии оценивания (максимум 100 баллов)

- **Блок 1. Промежуточные показы (40 баллов)**
 - **Показ 1 (10):** концепция, UX-сценарии, обоснование стека, схема данных, план PWA.
 - **Показ 2 (10):** рабочее ядро (операции + хранение), качество реализации, базовая структура проекта.
 - **Показ 3 (10):** категории/фильтры/аналитика, корректность расчетов, качество UI.
 - **Показ 4 (10):** целостный MVP: PWA-установка, офлайн-работа, стабильность.
- **Блок 2. Финальный продукт и техническая реализация (40 баллов)**
 - **Качество и стабильность MVP (15):** отсутствие критических багов, корректные расчеты, надежное хранение.
 - **Качество кода и архитектуры (15):** читаемость, структура, понятные слои, аккуратная работа с состоянием и данными, Git-практики.
 - **UI/UX и PWA-готовность (10):** удобный ввод, понятная навигация, соответствие PWA требованиям (manifest/SW), приемлемые метрики Lighthouse.
- **Блок 3. Документация и защита (20 баллов)**
 - **README и тех.описание (5):** запуск, архитектура, offline/PWA, ограничения и допущения.
 - **Презентация и демонстрация (10):** логика, ценность, ясное демо ключевых сценариев.
 - **Ответы на вопросы (5):** понимание offline-first, хранения данных, ограничений браузера и принятых решений.

Описание уровней оценки

- **Отлично (75–100):** все показы в срок; приложение удобно, стабильно, офлайн работает предсказуемо; PWA корректно устанавливается; код и документация высокого качества; возможны функции сверх MVP (повторяющиеся операции, импорт/экспорт, синхронизация).
- **Хорошо (55–74):** MVP полностью реализован; есть мелкие замечания по UX/стабильности; PWA и офлайн в целом работают.
- **Удовлетворительно (40–54):** базовые сценарии есть, но заметны баги/ограничения; офлайн/PWA реализованы частично; документация слабая.
- **Неудовлетворительно (0–39):** MVP не работает или отсутствуют ключевые требования (операции/хранение/офлайн/PWA); проект не готов к защите.